Software Design Analysis Report

A. Project Details

Project title	Online Doctor Appointment System (a web-based system that lets patients find doctors, view free time slots and book or cancel appointments, while doctors and the administrator manage them)
Team / roll numbers	Priyanshi - [Roll No.] [Team member 2] - [Roll No.] [Team member 3] - [Roll No.]
Project objective	To build a web-based system where a patient registers or logs in, searches doctors by specialization, selects an available time slot and books an appointment. The backend validates every booking and prevents double booking. Doctors view their appointments and update the status (Confirmed, Completed, Cancelled), and the administrator manages doctors, specializations and time slots.
Technology stack	HTML, CSS, JavaScript (front end), Node.js, Express.js (REST API), MySQL (database)

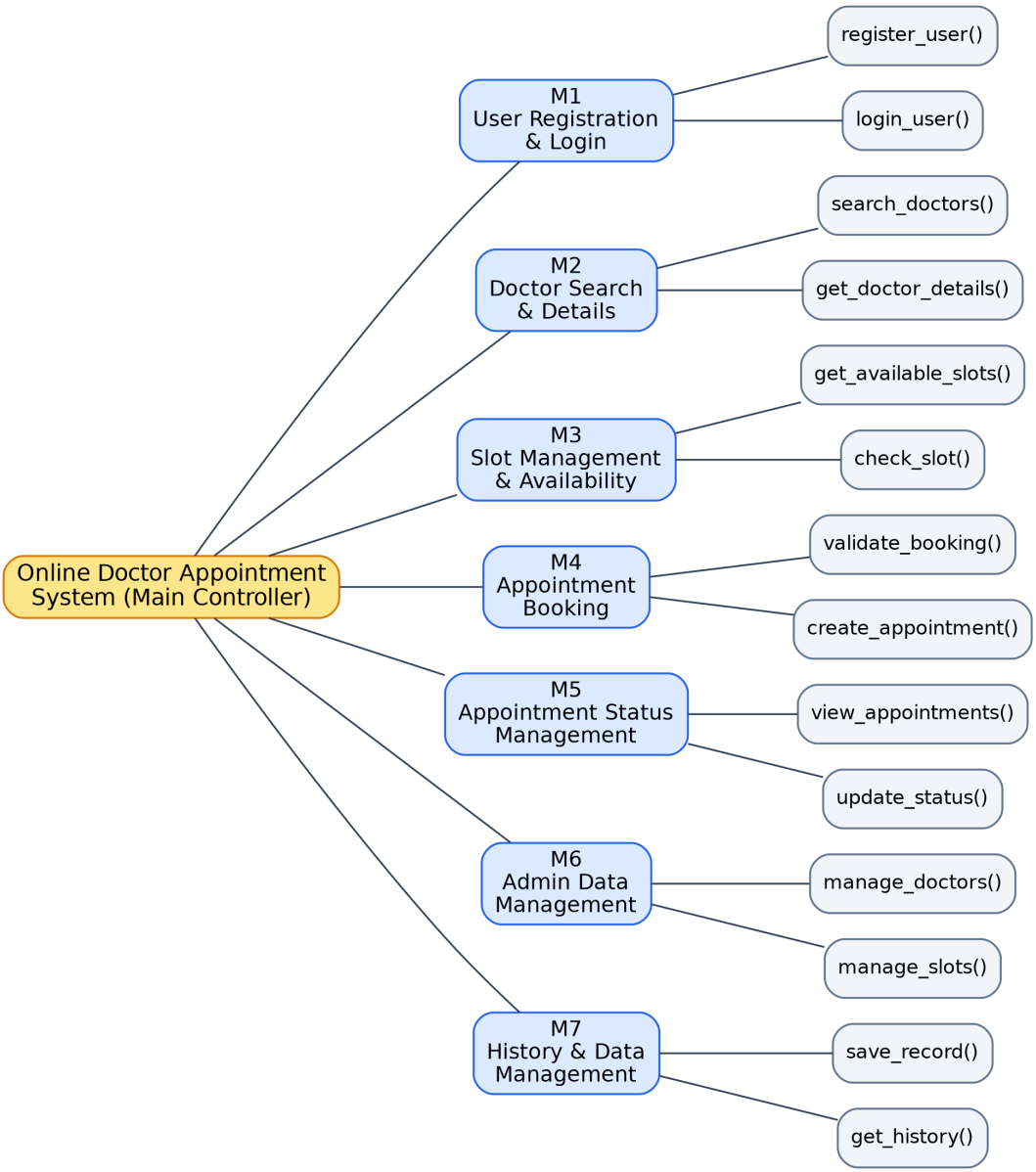
Note: the project follows the assigned topic (Online Doctor Appointment System); edit the content if your scope differs (e.g., video consultation or online payment).

B. Module Identification and Modularization

Total number of modules: 7. The system is divided along the stages of the booking process (identity, search, availability, booking, status handling, administration) plus one supporting module for history and data storage. Each module has one clear job, so modules can be developed, tested and replaced independently (for example, the slot-checking logic can change without touching login or search code).

Module ID	Module name	Main responsibility	Inputs / outputs	Depends on
M1	User Registration & Login	Register users, check credentials, start a session	In: email, password / registration form. Out: session token, user ID and role	M7 (user records)
M2	Doctor Search & Details	List doctors, filter by specialization, show fee and details	In: specialization / name filter. Out: list of doctors	M1
M3	Slot Management & Availability	Show free time slots, check and release a slot	In: doctor ID, date. Out: available slots, slot status	M2
M4	Appointment Booking	Validate the booking, create the appointment and lock the slot	In: patient ID, slot ID, reason. Out: appointment ID, confirmation	M1, M3
M5	Appointment Status Management	Doctor views appointments and updates status; cancelled slot is released	In: appointment ID, new status. Out: updated appointment	M4, M3
M6	Admin Data Management	Add or update doctors, specializations and time slots	In: doctor / slot details. Out: saved or updated records	M1 (admin role), M7
M7	History & Data Management	Store and retrieve records, show a patient's booking history	In: record, user ID. Out: stored data, appointment history	Database

Module hierarchy / decomposition diagram:



C. Design Strategy

Selected strategy: [ ] Top-Down [ ] Bottom-Up **[X] Hybrid**

Justification: The overall flow of the system (log in, search a doctor, choose a slot, book, manage status) is clear from the start, so the design began top-down by splitting the whole system into modules. However, the core booking part depends on ready-made, reusable components such as the Express.js routing and middleware, a MySQL driver and a password-hashing library, which are best built and tested bottom-up first. A hybrid strategy lets us decompose the problem clearly while reusing tested components and integrating them step by step, which reduces risk and development time.

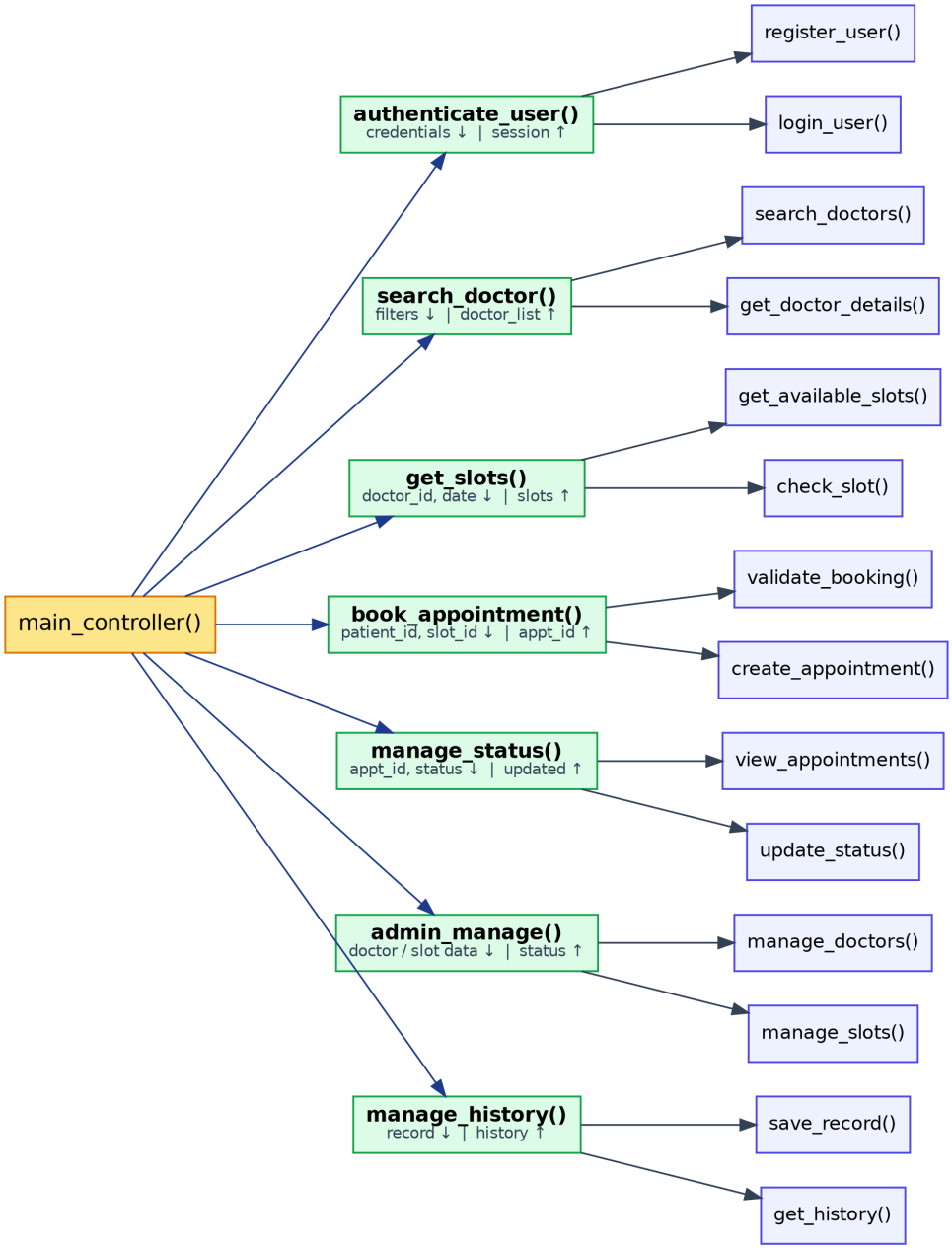
Evidence: Top-down: the system was first defined as one goal, 'let patients book doctor appointments online', then broken into 7 modules (M1-M7) and further into functions (see decomposition above). Bottom-up: low-level units (check_slot, validate_booking, create_appointment) were coded and unit-tested first, then combined into M3 and M4 and finally integrated with M1, M2, M5, M6 and M7 through the main controller.

D. Design Approach

Selected approach: [ ] Function-Oriented [ ] Object-Oriented **[X] Mixed**

Justification: The booking process is naturally a sequence of functions (validate login, search doctors, get slots, check slot, create appointment, update status), so a function-oriented design suits the processing part. The data and actors of the system (Patient, Doctor, TimeSlot, Appointment) are real-world entities with attributes and behaviour, so they are modelled as classes. Main functions: login_user(), search_doctors(), check_slot(), create_appointment(), update_status(). Main classes: Patient, Doctor, Specialization, TimeSlot, Appointment, Administrator (see UML class diagram).

Evidence - Structure chart: The main controller calls each module function. The second line inside each green box shows the data passed down to the function and the data returned up.



Explanation: main_controller() coordinates the whole process and is the only function that knows the order of execution. Each called function receives only the data it needs (data coupling) and returns a result, so modules do not call each other directly.

E. Coupling Analysis

Aim: low coupling. All interactions below pass only the required data through the controller.

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M1 - M2	Authenticated user ID and role	Data coupling (low)	Pass only the user ID and role, not the whole session object
M2 - M3	Selected doctor ID	Data coupling (low)	Pass only the doctor ID; the slot module does not know how the doctor list is built
M3 - M4	Slot ID and availability flag	Data coupling (low)	Pass only the slot ID; M4 re-checks availability through check_slot()
M4 - M5	Appointment object (ID, patient, slot, status)	Stamp coupling (moderate)	Pass only the appointment ID and the new status
M5 - M3	Slot ID to release after a cancellation	Data coupling (low)	Call release_slot(slot_id) only; no access to slot internals
M4 - M7	Appointment record to store	Data coupling (low)	Access the database only through AppointmentRepository methods
M6 - M7	Doctor / slot form object (name, fee, schedule)	Stamp coupling (moderate)	Use a small DTO with only the fields that are saved

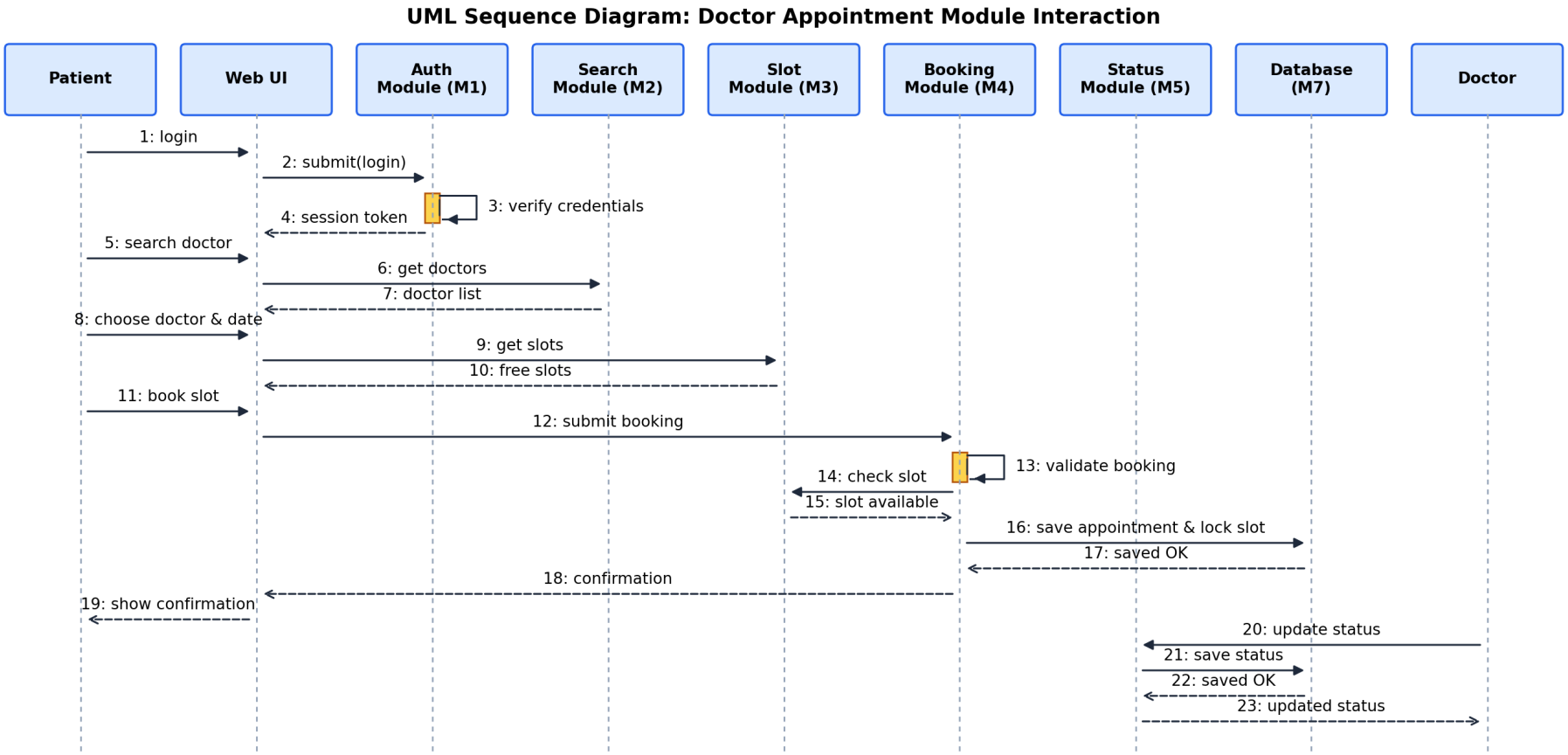
F. Cohesion Analysis

Aim: high cohesion. Each module has one main responsibility.

Module	Main responsibility	Cohesion type / level	Improvement, if needed
M1 Registration & Login	Check credentials and start a session	Functional (high)	None; keep password hashing in a separate helper
M3 Slot Management	Show free slots and check or release a slot	Functional (high)	Keep slot generation (admin) separate from availability checking
M4 Appointment Booking	Validate the request and create the appointment	Sequential (high): output of validation feeds creation	Keep notifications and payment outside this module (not in scope)
M5 Status Management	View appointments and change their status	Functional (high)	Call M3 to release a slot instead of duplicating that logic
M7 History & Data	Store records and return booking history	Communicational (medium): both use the appointment and patient tables	Separate data access and history into two classes if the module grows

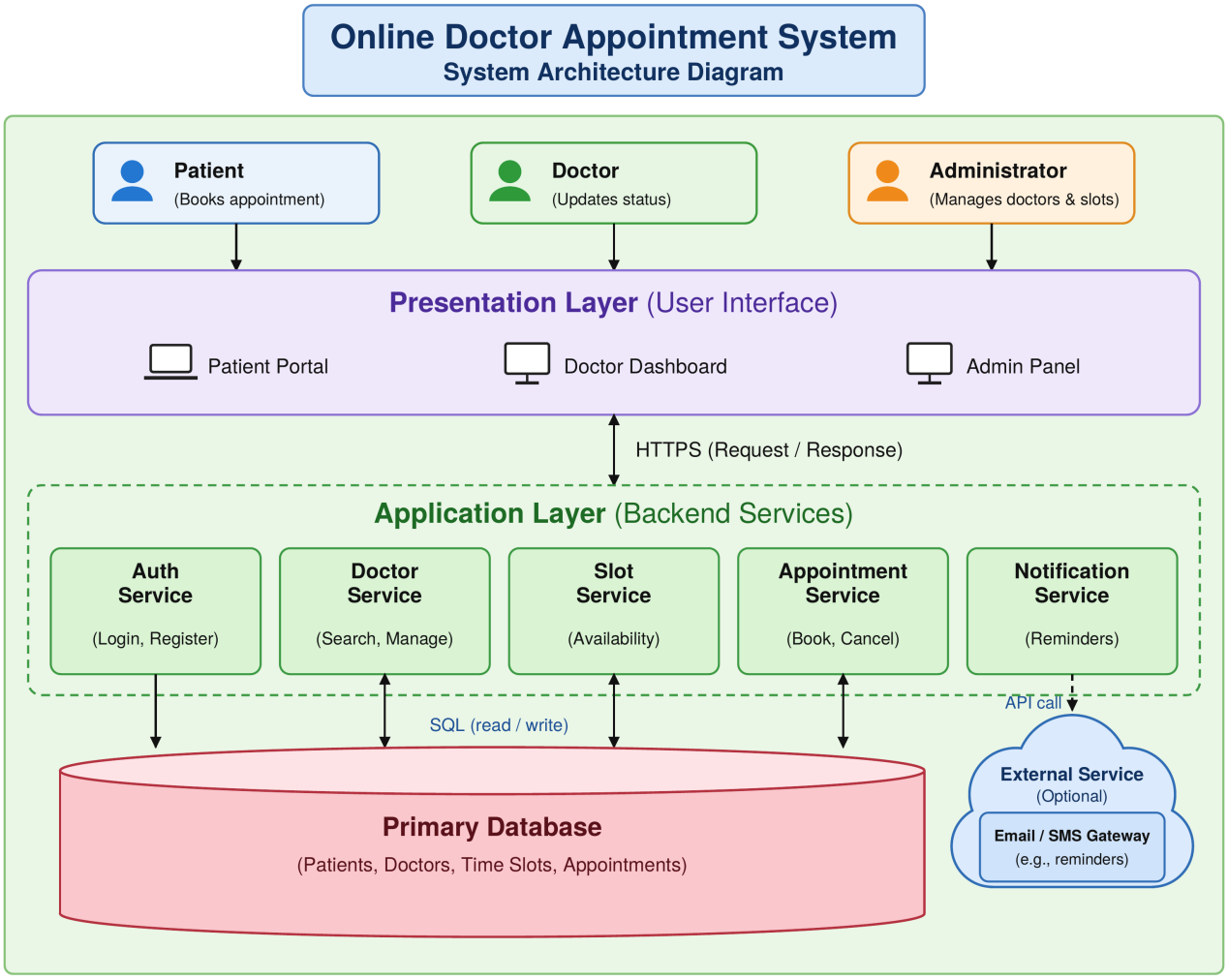
G. Module Interaction and Design Quality

Sequence diagram showing how the major modules exchange information for one booking and one status update:



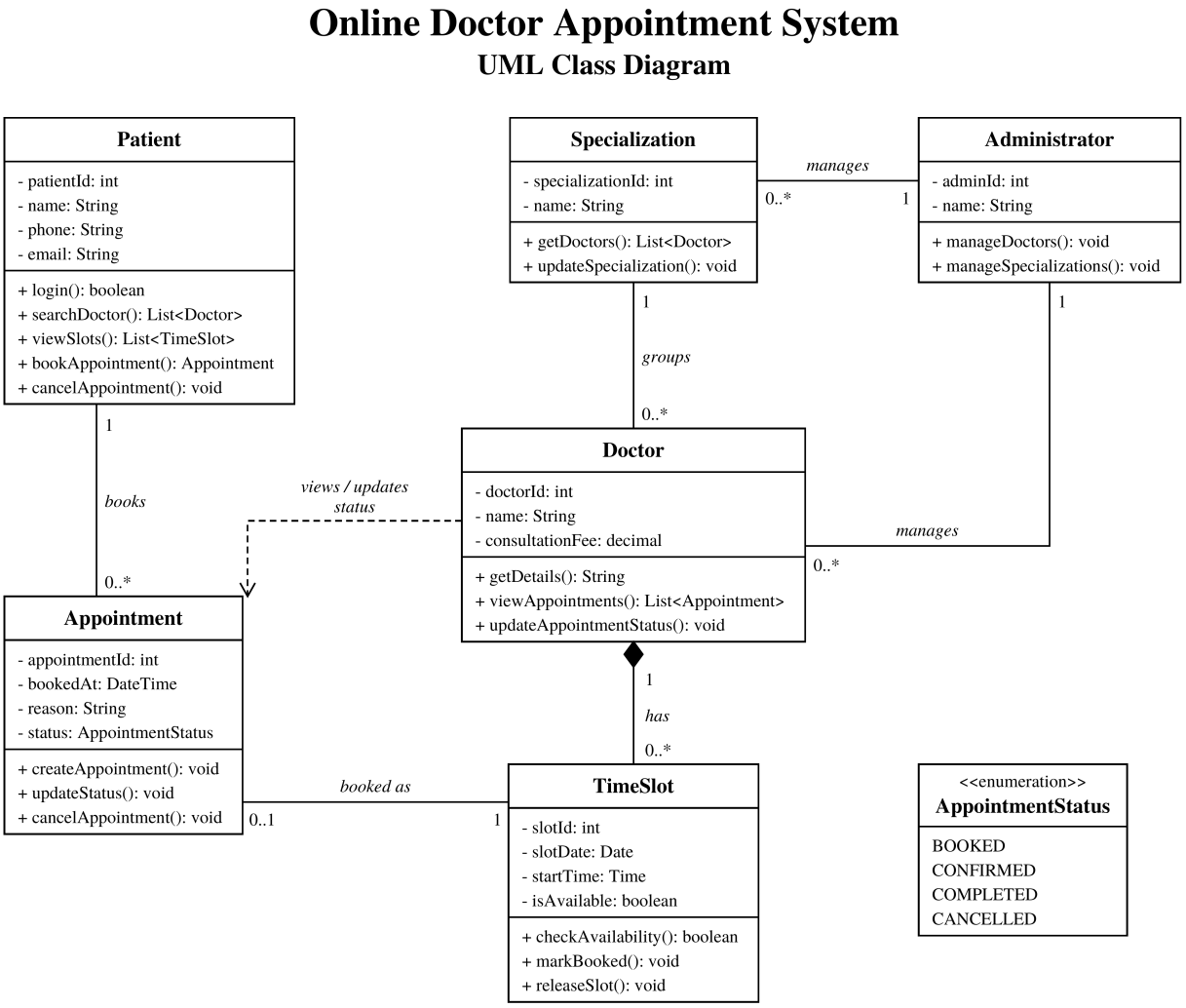
Design quality summary: modules communicate in a single direction along the booking process, coupling is mostly data coupling, and every module has one clear responsibility (functional or sequential cohesion). This makes the system easy to test, maintain and extend (e.g., adding email or SMS reminders only needs a new notification module that is called after M4 and M5).

Annexure 1: System Architecture Diagram



Legend: rounded box = component/service; cylinder = database/storage; cloud = external service (optional); solid arrow = data flow; dashed arrow = dependency; dashed boundary = application layer.

Annexure 2: UML Class Diagram



Annexure 3: UML Activity Diagram

UML Activity Diagram: Appointment Booking Workflow

